



R Data Structures: Building Blocks for Data Analysis

A comprehensive guide to mastering the fundamental data structures that power statistical computing and data analysis in R.

The Foundation: Atomic Vectors

Atomic vectors represent the simplest and most fundamental data structure in R. Think of them as the basic building blocks upon which all other data structures are constructed. They form the backbone of data manipulation in R.

Homogeneous Structure

All elements must be of the same data type—numeric, character, logical, or complex. This constraint ensures data integrity and enables efficient vectorised operations.

Ordered Collection

Elements maintain a specific order, allowing for indexed access. Each position in the vector can be referenced using its index number, starting from 1.

Single Dimension

Vectors are one-dimensional structures, conceptually similar to a single row or column of data in a spreadsheet. They represent the simplest form of organised data.

Creating Vectors: The `c()` Function

The `c()` function is your primary tool for combining individual values into a vector. This versatile function "combines" or "concatenates" elements together, creating a single vector from multiple components. It's one of the most frequently used functions in R programming.

Numeric Vector

```
numeric_vector <- c(10, 25, 15, 30, 45)
print(numeric_vector)
# Output: [1] 10 25 15 30 45
```

A sequence of numbers that can be used for mathematical operations and statistical analysis.

Character Vector

```
character_vector <- c("Apple",
                      "Banana", "Cherry", "Date")
print(character_vector)
# Output: [1] "Apple" "Banana"
          "Cherry" "Date"
```

Text data enclosed in quotation marks, perfect for storing names, categories, or labels.

Logical Vector

```
logical_vector <- c(TRUE, FALSE, TRUE,
                    TRUE, FALSE)
print(logical_vector)
# Output: [1] TRUE FALSE TRUE
          TRUE FALSE
```

Boolean values representing TRUE or FALSE conditions, essential for logical operations and filtering.

Beyond Vectors: Lists

Lists represent a significant advancement in data structure flexibility. Unlike atomic vectors, lists are **heterogeneous**, meaning they can contain elements of different data types within the same structure. This makes lists incredibly versatile for storing complex, mixed-type data.

01

Ordered Collection

Elements maintain their position and can be accessed by index number, just like vectors.

02

Any Data Type

Each element can be any R object—vectors, matrices, data frames, or even other lists.

03

Nested Structure

Lists can contain other lists, creating hierarchical data structures of arbitrary complexity.

Example: Creating a List

```
my_list <- list(  
  name = "Alice",  
  age = 30,  
  scores = c(85, 92, 78),  
  passed = TRUE  
)  
print(my_list)
```

This list combines character, numeric, vector, and logical data types into a single structure, perfect for representing complex real-world data.

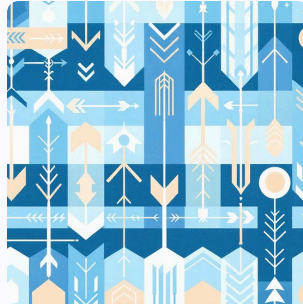
Adding Dimensions: Matrices

Matrices extend the concept of vectors into two dimensions, organising data into rows and columns. Like atomic vectors, matrices are **homogeneous**—all elements must be of the same type. This constraint enables efficient mathematical operations across the entire structure.



Creation Function

The `matrix()` function creates matrices from vectors, specifying dimensions with `nrow` and `ncol` parameters.



Filling Order

By default, matrices fill column-wise. Use `byrow = TRUE` to fill row-wise instead.



Mathematical Operations

Support element-wise operations and matrix algebra including multiplication, transpose, and inverse calculations.

Example: Creating a Matrix

```
my_matrix <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
print(my_matrix)
```

Output:

```
#      [,1] [,2] [,3] [,4]
# [1,]  1   2   3   4
# [2,]  5   6   7   8
# [3,]  9  10  11  12
```

This creates a 3×4 matrix filled with numbers 1-12, arranged row by row. The matrix has 3 rows and 4 columns, with 12 total elements.

Higher Dimensions: Arrays

Arrays represent the generalisation of matrices to more than two dimensions. Whilst matrices are limited to two dimensions (rows and columns), arrays can have any number of dimensions—1D, 2D, 3D, or even higher. This makes arrays perfect for representing complex multi-dimensional data such as time series, image data, or scientific measurements.



Multi-Dimensional

Can have 1, 2, 3, or more dimensions.
Each dimension represents a different aspect of the data structure.



Homogeneous Elements

Like vectors and matrices, all elements must be of the same data type for efficient storage and operations.



Flexible Dimensions

Created using the `array()` function with a `dim` parameter specifying size of each dimension.

Example: Creating a 3D Array

```
my_array <- array(1:24, dim = c(2, 3, 4))  
print(my_array)
```

```
# Creates a 3D array with dimensions:  
# - 2 layers (first dimension)  
# - 3 rows (second dimension)  
# - 4 columns (third dimension)  
# Total elements:  $2 \times 3 \times 4 = 24$ 
```

This three-dimensional array can be visualised as 2 layers, each containing a 3×4 matrix. Arrays are particularly useful for storing data with natural multi-dimensional structure, such as colour images (height × width × colour channels) or time series data (time × variables × observations).

The Workhorse: Data Frames

Data frames are arguably the most important data structure for statistical analysis in R. They combine the rectangular organisation of matrices with the flexibility of lists, creating the perfect structure for real-world tabular data. Data frames are the go-to choice for most data analysis tasks.



Rectangular Structure

Organised into rows (observations) and columns (variables), similar to a spreadsheet or database table.



Heterogeneous Columns

Each column can have a different data type—numeric, character, logical, or factor—while maintaining consistent row structure.



Named Columns

Columns have descriptive names, making data easier to reference and understand. Rows can also have names.

Created using the `data.frame()` function, data frames are the standard format for importing and exporting data from CSV files, databases, and other sources. They're optimised for the type of mixed-type data commonly encountered in statistical analysis.

Data Frame Dimensions and Examples

Understanding the dimensions of data frames is crucial for effective data manipulation. A data frame has two primary dimensions: **rows** (representing individual observations or records) and **columns** (representing variables or features). Each column can have a different data type, making data frames incredibly versatile.

Example: Student Data Frame

```
student_data <- data.frame(  
  Name = c("Alice", "Bob", "Charlie"),  
  Age = c(21, 22, 20),  
  Major = c("Math", "Physics", "Chemistry"),  
  GPA = c(3.8, 3.5, 3.9)  
)
```

```
print(student_data)  
# Output:  
#   Name Age  Major GPA  
# 1 Alice 21   Math 3.8  
# 2  Bob 22 Physics 3.5  
# 3 Charlie 20 Chemistry 3.9
```

3

Rows (Observations)

Each row represents a single student in our dataset

4

Columns (Variables)

Four different pieces of information recorded for each student

12

Total Elements

3 rows × 4 columns = 12 individual data points

This data frame demonstrates the power of heterogeneous structure: **Name** is character, **Age** is numeric, **Major** is character, and **GPA** is numeric. Each column maintains its own data type whilst being part of a unified structure.

Accessing Data: Subsetting

Subsetting is the process of extracting specific elements from data structures. Different data structures require different subsetting methods, each designed to efficiently access elements in their particular organisation. Mastering subsetting is essential for data manipulation in R.



Vectors: Single Brackets

Use square brackets `[]` with indices to access elements by position.

```
# Access 2nd element
numeric_vector[2]
# Returns: 25

# Access multiple elements
numeric_vector[c(1, 3, 4)]
# Returns: [1] 10 15 30
```



Lists: Double Brackets

Use double square brackets `[[ ]]` to extract elements, or `$` for named elements.

```
# Extract by index
my_list[[1]]
# Returns: "Alice"

# Extract by name
my_list$name
# Returns: "Alice"

# Extract nested element
my_list$scores[2]
# Returns: 92
```



Matrices & Data Frames

Use `[, ]` with row and column indices. Omit indices to select all rows or columns.

```
# Access specific cell
my_matrix[1, 2]
# Returns: 2

# Access entire row
student_data[3, ]
# Returns 3rd student's data

# Access specific column
student_data[, "GPA"]
# Returns all GPAs
```



Arrays: Multiple Brackets

Use `[,, ]` for 3D arrays, adding more commas for additional dimensions.

```
# Access element in 3D array
my_array[1, 2, 3]
# Returns element at position (1,2,3)

# Access 2D slice
my_array[1, , ]
# Returns first layer as a matrix
```

Mastering R Data Structures: Your Analytical Powerhouse

Understanding R's data structures is fundamental to becoming proficient in data analysis and statistical computing. Each structure serves a specific purpose, and choosing the right one for your data can dramatically improve code efficiency and clarity.



Vectors

The fundamental building blocks for all data structures. Use for homogeneous, one-dimensional data and efficient vectorised operations.



Lists

For flexible, heterogeneous collections containing mixed data types. Perfect for complex, nested data structures.



Matrices & Arrays

For multi-dimensional, homogeneous data requiring mathematical operations across dimensions.



Data Frames

The go-to structure for tabular, mixed-type data representing real-world observations and variables.

By mastering these core data structures, you'll be equipped to handle virtually any data analysis challenge in R. Each structure has its strengths, and understanding when to use each one is key to writing efficient, readable, and effective R code for statistical computing.